Power-Demand-Forecast プロジェクト憲法

核心原則 (Core Principles)

I. テスト駆動開発の徹底 (TDD優先)

原則

- すべての機能実装において、テストを先に作成し、仕様に対する検証を必須とする
- テストが失敗することを確認してから実装を開始する (Red-Green-Refactorサイクル)
- 単体テスト、統合テスト、契約テストを段階的に実施する
- テストカバレッジ目標値の設定値保証を常に目指した仕様を定期的に行う

理由

- 仕様の認識を早期に発見し、手戻りコストを最小化する
- リファクタリングやAI支援を安全に実施し、開発的負債を防ぐ
- チキンエッグ問題を回避し、コードの設計を明確にする
- 複雑な要件を少しずつ分割してテスト可能な単位で実装する安定性を保証する

適用例

- モデル学習スクリプトでは、期待精度（R² > 0.80）を満たすテストケースとして定義
- データ処理パイプラインでは、入力値の型と範囲をpandera等でチェック
- API連携では、Open-Meteo APIのレスポンスをmocks使用テストで検証

II. セキュリティファースト原則

原則

- セキュリティ要件を機能要件より優先する
- 機密データ（APIキー、認証トークン、個人情報）の平文保存を禁止
- すべての外部APIエンドポイントはHTTPSのみを許可する
- 環境変数による機密情報管理を徹底する
- セキュリティパッチ適用を最優先で実施する

理由

- 後からのセキュリティ対策は設計変更やコスト増大を伴う
- データ漏洩は事業継続性に致命的な損害を与える
- 電力需要データ・位置情報はインフラ関連情報であるため慎重な扱いが必要
- 信頼性エンジニアリングとも密接する

適用例

- GitHub Actions でGitSecretsを使用し、環境変数でAPI情報を管理
- .gitignore で .env ファイル（.gitignore で除外）を使用
- Open-Meteo APIの認証はHTTPSエンドポイントを使用
- パスワード、トークンなどはログ出力や出力に含まない

III. パフォーマンス定量化原則

原則

- パフォーマンス期待値を定量化し、受入基準に組み込む
- すべてのモデル学習・予測処理に実行時間とメモリ使用量の上限を設定
- 推奨指標（MAE, RMSE, MAE）を明確に定義し、開発者・運用担当者間でアラート
- Github Actions実行時間の制限を考慮した最適化を行う

理由

- 環境は「高速だ」ではなく、測定可能な目標を設定することで改善が可能になる
- 本番環境（Github Actions）での実行可能性を保証する
- ユーザ・体験（ダッシュボードの応答速度）を維持する
- リソース上ユス（CUCDE実行時間）を最適化する

適用例

- LightGBMパラメータ探索時、30秒（Github Actions環境）
- Kaggle学習時間 < 60時（Github Actions環境）
- クリンエネ・APIの応答時間 < 2秒（ローカルHTTPサーバ）
- ダスコア・ < 0.80（開発者調査CSVの記録作成）
- メモリ使用量: float32型を使用し、メモリ使用量を約50%削減

IV. データ保護の徹底

原則

- 機密データ・個人情報を保護し、暗号化または匿名化なしに保存しない
- データ学習は公開可能な匿名・匿名可能性を保証する
- 個人情報を含む可能性のあるデータは匿名化処理を実施
- データのバックアップは定期的に実施し、整合性を保証する

理由

- データ漏洩によるおよびリスクとプライバシ保護を守る
- GDPR, 個人情報保護法などのコンプライアンス要件に対応
- データ自体を生と可能性を保証する
- 信頼性データ・の機密性を維持する

適用例

- 気温データはOpen-Meteo APIから取得し、公開データのみを使用
- ローカルスケーンにはセッションIDのみを保（個人情報なし）
- サーバ・ログには個人を特定できる情報を含まない
- データベース使用時は暗号化スキーマを採用

V. 依存関係の再現性保証

原則

- 外部依存パッケージはバージョンを固定し、requirements.txt で厳密に管理
- Python実行環境のバージョンを明示する (Python 3.10.11)
- Github Actionsのワークフローファイルもテストで固定する
- 依存パッケージの更新は計画的にテストし、管理戦略を評価する

理由

- 環境差によるまれにしか動作を防ぐ
- 機械学習モデルの再現性を保証する
- デプロイ時のトラブルシューティングを容易にする
- CUCDE環境ではローカル環境の一致性を維持する

適用例

- requirements.txt ファイルでパッケージ名のみ で固定（例: lightgbm~=3.3.5）
- Github Actions で python-version: '3.10.11' を明記
- Github Actionsのワークフローで pip をのぞくパッケージ以外プ使用
- 依存パッケージ更新時はまずテストを行い、機能変化がないことを確認

VI. 仕様実装乖離の検出

原則

- 仕様と実装の乖離をレビューで検知・是正する
- すべてのPull Request に仕様へのリンクを含める
- 仕様変更時は仕様書を先に修正し、同期する
- ドキュメント（README, 仕様書）とコードの整合性を定期的に検証する

理由

- 仕様と実装の乖離は長期的でスパゲティ状態を生じさせる原因
- 長期的なプロジェクト運用において、チキュンエッ状態を反映しなくなる問題を防ぐ
- コードレビュー効率を上げる
- チーム間のコミュニケーションを円滑する

適用例

- Pull Requestテンプレートに「関連仕様」セクションを追加
- 仕様書（docs/仕様書.md）とREADMEの整合性をエ定期チェック
- Github Actionsワークフローの定期実行（docs/DOCUM_guide.md を同時実行
- モデル精度関連の変更時は、仕様書とコードの両方を更新

ブランチ戦略

仕様ブランチ（mainブランチから派生）

```
git checkout main
git checkout -b <番号>-<説>/<名>
# 例: git checkout -b 001-Power-Demand-Forecast
```

用途

- 新機能の仕様策定
- アーキテクチャ設計
- ドキュメント作成

マージ条件

- 仕様レビュー完了
- 関連チキュンエッ更が完了

実装ブランチ（仕様ブランチから派生）

```
git checkout 001-Power-Demand-Forecast
git checkout -b feature/impl-001-Power-Demand-Forecast
```

用途

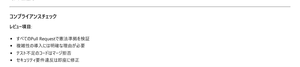
- 実装作業
- テスト作成・実行
- コードレビュー

マージ条件

- すべてのテスト通過
- コードレビュー承認
- パフォーマンス要件達成
- セキュリティチェック完了

開発ワークフロー

作業順序（厳守）



1. **要件確認**: 要件の曖昧性を確認し、事業価値を確認
2. **仕様作成**: 機能仕様書を作成（specs/番号-一機能名/spec.md）
3. **実装計画**: アーキテクチャ技術選定を文書化（specs/番号-一機能名/plan.md）
4. **テスト作成**: 受入テスト基準を作成（specs/番号-一機能名/tasks.md）
5. **テスト作成**: TDDに従い、失敗するテストを作成
6. **実装**: テストが通るまで実装
7. **検証**: すべてのテストと品質基準を確認
8. **レビュー**: Pull Requestを作成し、レビュー承認を得る
9. **マージ**: mainブランチにマージ

フロントエンドとバックエンドの同時起動

開発方針:

- フロントエンド（ダッシュボード）とバックエンド（Python HTTPサーバ）を同時に起動するコードを生成
- 正常に動作するまで繰り返し検証し、エラー修正を完了する

実装例

```
# ターミナル1: 起動 (バックエンド)
./r.AI
py -3.10 server.py
# アクセス可能な箇所 (フロントエンド): http://localhost:8002/
```

Mermaid図の1行原則のベストプラクティス

推奨事項:

1. **graphviz使用時の注意**
   - 日本語を避けるか、flowchart/graphがより安全
   - label 構文は単純化、代わりにノードを用
   - 矢印(->)は2文字の矢印でラベルに使用できない
2. **日本語対応**:
   - flowchart, graph, sequenceDiagramは日本語完全対応
   - ノードIDは、エンプ内ら、Note内は自動用可能
   - gitGraphは英文日本語コントットーでに対応していない
3. **推奨構文**:
   - フローチャート: flowchart TB + subgraph
   - 時系列: sequenceDiagram
   - 状態遷移: stateDiagram-v2

ガバナンス

憲法の優先順位

原則:

- この憲法はすべてのプロジェクト慣行より優先される
- 憲法と矛盾する慣行は即座に是正される
- 憲法改修時に関係する各担当、プロジェクトリーダ-が議論・評価を行う

憲法の改訂手順

プロセス

1. 改訂提案をGithub Issueで起票
2. 影響範囲分析を実施
3. レビュー承認を取得
4. バージョン番号更新（セマンティックバージョニング）
   - MAJOR: 後方互換性のない原則の削除・再定義
   - MINOR: 新原則/セクション追加または重要な拡張
   - PATCH: 明確化、文言修正、非セマンティクな修正
5. 修行計画を文書化
6. 関連テンプレートとドキュメントを同期更新
7. 最終改訂日を更新

品質ゲート

必須チェック項目

- [ ] すべてのテストが通過している
- [ ] パフォーマンス期待を達成している
- [ ] セキュリティチェックが完了している
- [ ] ドキュメントが最新状態である
- [ ] 依存関係のライセンス確認されている
- [ ] 仕様と実装が一致している

重大変更の定義

- 公開APIの変更
- データ・フロの変更
- 機密関連の変更
- セキュリティポリシーの変更
- 依存パッケージのメジャーバージョンアップ

重大変更の承認プロセス

1. 影響範囲のドキュメント作成
2. 影響範囲の分析
3. レビュー承認（最低1名）
4. ステージング環境でのテスト
5. 本番適用

コンプライアンスチェック

レビュー検査

- すべてのPull Requestで憲法準拠を検証
- 違反性の疑入には明確な理由の必要
- テストで正当ハ証すべ違反
- セキュリティ要件違反は即座に修正

ランダム開発ガイダンス

参照ドキュメント:

- 開発時の詳細ガイダンスは docs/完成仕様書.md を参照
- デプロイ手順は docs/deploy_guide.md を参照
- ロ-カル環境セットアップは docs/開発手順書.md を参照